asignacion

«desnormalizada» RankingPrecalculado
private idDonacion: UUID
private puestos: List<PuestoRanking>
private fechaCalculo: LocalDateTime
private nuevo: boolean {transient}
public de(idDonacion, resultado, fechaCalculo): RankingPrecalculado
public resultado(algoritmos: List<String>): ResultadoMatchmaking
public sinCandidatas(): boolean

«embeddable» PuestoRanking
private algoritmo: String
private posicion: int
private resultado: ResultadoAsignacion

«embeddable» ResultadoAsignacion
private idEntidad: UUID
private puntaje: double

«valor» ResultadoMatchmaking
rankings: Map<String, List<ResultadoAsignacion>>
public ranking(algoritmo: String): List<ResultadoAsignacion>
public completadoCon(algoritmos: List<String>): ResultadoMatchmaking
public coincidencias(): List<ResultadoAsignacion>
public huboCoincidencias(): boolean
public sinCandidatas(): boolean

«desnormalizada» RecepcionDiariaEntidad
private clave: Clave
private cantidad: long

«embeddable» RecepcionDiariaEntidad.Clave
private idEntidad: UUID
private fecha: LocalDate

Tablas de lectura de la Entrega 5: derivables de donacion y necesidad.
RankingPrecalculado: lo escribe el proceso nocturno de matchmaking; la bandeja lo lee tal cual. Un puesto por algoritmo y posición: un algoritmo nuevo no cambia el esquema. Las coincidencias se derivan de los rankings. Un algoritmo sin entidades no deja puestos: resultado[algoritmos] lo agrega con la lista vacía, así la bandeja muestra todos y no hay coincidencias si a uno no le quedó ninguna.
RecepcionDiariaEntidad: donaciones asignadas por entidad y día. Se actualiza en la misma transacción que la asignación y se reconcilia cada noche desde la tabla donacion.

categoria

«embeddable» Subcategoria
private tipo: String
private tipoNormalizado: String
public tiposDistintos(subcategorias: Collection<Subcategoria>): List<String>
public estaVacia(): boolean
public coincideCon(otra: Subcategoria): boolean
public estaMencionadaEn(texto: String): boolean

Compara por tipoNormalizado (NormalizadorTexto: minúsculas, sin acentos, espacios colapsados), que es también la columna indexada para buscar. El tipo se guarda sin espacios en los extremos; tiposDistintos da una vez cada subcategoria (las categorias que se informan a Incentivos).

donacion

bien

BienGenerico
BienConEstado
private esNuevo: boolean
BienPerecible
private fechaVencimiento: LocalDate

«abstracta» Bien
protected id: UUID
protected subcategoria: Subcategoria
protected descripcion: String
protected foto: String
protected cantidad: int
protected unidad: String
public tipo(): TipoBien
public vencimiento(): Optional<LocalDate>
public nuevo(): Optional<Boolean>
public claveDeSegmentacion(): ClaveDeSegmentacion
protected criterioDeSegmentacion(): Object

«valor» Bien.ClaveDeSegmentacion
subcategoria: Subcategoria
tipo: TipoBien
criterio: Object

«enum» TipoBien
PERECIBLE
CON_ESTADO
GENERICO

«enum» ProveedorLogistica
DONATRACK
EXTERNA

«valor» Asignacion
idEntidad: UUID
fecha: LocalDate
NINGUNA_Asignacion
public existe(): boolean

CargaDonacion
private idDonante: UUID
private descripcion: String
private bienes: List<Bien>
public segmentar(): List<Donacion>

Donacion
protected bienes: List<Bien>
protected subcategoria: Subcategoria
protected estadoNombre: String
protected historialEstados: List<CambioEstado>
protected idDonante: UUID
protected idEntidadBeneficiaria: UUID
protected descripcion: String
protected fechaDonacion: LocalDateTime
protected fechaAsignacion: LocalDate
protected proveedorLogistica: ProveedorLogistica
protected idSeguimientoLogistica: String
protected fechaBaja: LocalDateTime
private asignacionGuardada: Asignacion {transient}
public getEstado(): EstadoDonacionBase
public cambiarEstado(estadoDestino, nombreTransicion, justificacion): void
public cambiarEstadoManualmente(estadoDestino, justificacion): void
public asignarA(entidad: EntidadBeneficiaria): void
public asignacion(): Asignacion
public cambioLaAsignacion(): boolean
public despacharA(proveedor: ProveedorLogistica, idSeguimiento: String): void
public iniciarTraslado(justificacion: String): boolean
public registrarEntrega(fechaEntrega: LocalDateTime, justificacion: String): boolean
public registrarEntregaFallida(justificacion: String): boolean
public darDeBaja(): void
public agregarBien(bien: Bien): void
public reemplazarBien(idBien: UUID, nuevo: Bien): void
public tieneBienesVencidosAl(fecha: LocalDate): boolean
public exigirSinBienesVencidosAl(fecha: LocalDate): void
public estaEnEstado(nombreEstado: String): boolean
public cantidadTotal(): int
+marcarAsignacionGuardada(): void +@PostLoad, @PostPersist, @PostUpdate»

Baja lógica: fecha_baja + @SQLRestriction, una donación dada de baja no aparece en ninguna consulta.
Los avisos de logística son idempotentes: iniciarTraslado / registrarEntrega / registrarEntregaFallida devuelven false si el aviso ya se había procesado.
A ASIGNACION_REALIZADA solo se llega con asignarA (el cambio manual la rechaza).

eventos

«evento» DonacionCambioDeEstado
idDonacion: UUID
estadoPrevio: String
estadoNuevo: String

«evento» DonacionAsignada
idDonacion: UUID
idDonante: UUID
idEntidad: UUID

«evento» DonacionEntregada
idDonacion, idDonante, idEntidad: UUID
subcategoria: Subcategoria
cantidad: int
fechaEntrega: LocalDateTime

«evento» DonacionDadaDeBaja
idDonacion: UUID

«evento» DonacionesRegistradas
idDonante: UUID
cantidadBienes: int
categorias: List<String>
cuando: LocalDateTime

estado

EnTrasladoState
protected validarTransicion(estadoDestino: String, justificacion: String): void
EntregadaState
EntregaFallidaState
VencidaState
EnDepositoState
AsignacionRealizadaState
ListaParaEntregarState

«abstracta» EstadoDonacionBase

EstadosDonacion
public EN_DEPOSITO, ASIGNACION_REALIZADA, ... : String
public inicial(): EstadoDonacionBase
public desde(nombre: String): EstadoDonacionBase
public desdeONull(nombre: String): EstadoDonacionBase

Transiciones válidas (estado inicial: EN_DEPOSITO):
EN_DEPOSITO → ASIGNACION_REALIZADA | VENCIDA
ASIGNACION_REALIZADA → LISTA_PARA_ENTREGAR
LISTA_PARA_ENTREGAR → EN_TRASLADO
EN_TRASLADO → ENTREGADA | ENTREGA_FALLIDA
ENTREGA_FALLIDA → EN_DEPOSITO | EN_TRASLADO
ENTREGADA y VENCIDA son finales
ENTREGA_FALLIDA sin justificación → FaltaJustificacionException
ASIGNACION_REALIZADA a mano → TransicionReservadaException

CambioEstado
private id: UUID
private estadoPrevioNombre: String
private estadoNombre: String
private nombreTransicion: String
private justificacion: String
private fechaHora: LocalDateTime
public de(estadoPrevio, estado, nombreTransicion, justificacion): CambioEstado
public getEstadoPrevio(): EstadoDonacionBase
public getEstado(): EstadoDonacionBase

contacto

Email
public canal(): Canal
private normalizarValor(): void +@PrePersist, @PreUpdate»
Telefono
public canal(): Canal
Whatsapp
public canal(): Canal

«abstracta» MedioDeContacto
protected id: UUID
protected valor: String
protected predeterminado: boolean
public canal(): Canal
public marcarComoPredeterminado(valor: boolean): void
public esDelMismoTipoQue(otro: MedioDeContacto): boolean

«enum» Canal
EMAIL
SMS
WHATSAPP

«interface» Persistable
UUID
public getId(): UUID
public isNew(): boolean

Base de las raices de agregado (Entrega 5).
id asignado por la aplicación + isNew():
sin SELECT previo a cada INSERT.
version: bloqueo optimista (409 si dos pedidos modifican lo mismo a la vez).
eventos: se publican al guardar el agregado.

«abstracta» EntidadPersistible
protected id: UUID
protected version: long «@Version»
protected nueva: boolean {transient}
private eventos: List<Object> {transient}
public isNew(): boolean
protected registrarEvento(evento: Object): void
public eventosPendientes(): List<Object> «@DomainEvents»
protected limpiarEventos(): void «@AfterDomainEventPublication»
~marcarComoPersistida(): void «@PostLoad, @PostPersist»

estado

CatalogoDeEstados
private porNombre: Map<String, Supplier<E>>
public desde(nombre: String): E

«abstracta» Estado
private nombre: String
private transiciones: Map<String, Supplier<E>>
public transicionarA(estadoDestino: String, justificacion: String): E {final}
protected validarTransicion(estadoDestino: String, justificacion: String): void
public nombre(): String
public es(nombreEstado: String): boolean

Base común de los estados de la donación y de la persona donante (patrón State). Cada estado declara a dónde puede pasar; validarTransicion es el gancho para sus reglas propias. Se guarda solo el nombre.
transición no declarada → TransicionDeEstadoIlegalException
nombre desconocido → EstadoDesconocidoException

asignarA()

donante

Representante
private id: UUID
private nombre: String
private apellido: String
private email: String
private contactos: List<MedioDeContacto>
~normalizarEmail(): void +@PrePersist, @PreUpdate»

PersonaJuridica
private razonSocial: String
private tipo: TipoPersonaJuridica
private rubro: String
private representantes: List<Representante>
public agregarRepresentante(representante: Representante): void
public actualizarDatosDeOrganizacion(razonSocial, rubro, tipo): void

«enum» TipoPersonaJuridica
GUBERNAMENTAL
ONG
EMPRESA
INSTITUCION

«abstracta» PersonaDonante
protected documento: Documento
protected direccion: Direccion
protected estadoNombre: String
protected ultimaInteraccion: LocalDateTime «sin versión»
protected ultimoAvisoInactividad: LocalDateTime
protected email: String {unique}
protected contactos: List<MedioDeContacto>
public getEstado(): EstadoDonante
public cambiarEstado(estadoDestino: String, justificacion: String): void
public estaBloqueada(): boolean
public registrarInteraccion(): void
public reemplazarContacto(medio: MedioDeContacto): void
public reemplazarContactos(nuevos: List<MedioDeContacto>): void
public definirContactoPredeterminado(medio: MedioDeContacto): void
public getMedioContactoPredeterminado(): MedioDeContacto
public cambiarDireccion(direccion: Direccion): void
public cambiarDocumento(documento: Documento): void
public tipoPersona(): TipoPersona
public actualizarNombre(nombre: String): void
private normalizarEmail(): void +@PrePersist, @PreUpdate»

email: identidad de la persona.
Se fija en el alta y no cambia después.
Se guarda en forma canónica (NormalizadorEmail: trim, NFC, minúsculas) y la base garantiza que sea único.
Sin setters: cada dato que se puede cambiar tiene su método; las listas son de solo lectura.
ultimaInteraccion queda fuera del bloqueo optimista (@OptimisticLock(excluded)): dos donaciones simultáneas de la misma persona se guardan las dos; cualquier otro cambio sube la versión.

PersonaHumana
private nombre: String
private apellido: String
private fechaNacimiento: LocalDate
private genero: Genero
public actualizarDatosPersonales(nombre, apellido, fechaNacimiento, genero): void

«enum» Genero
MASCULINO
FEMENINO
NO_BINARIO
OTRO
PREFIERO_NO_DECIR

«enum» TipoPersona
HUMANA
JURIDICA

«embeddable» Documento
private tipo: String
private numero: String
public de(tipo: String, numero: String): Documento
public deOrganizacion(tipo: String, numero: String): Documento

«enum» TipoDocumento
DNI
LC
LE
CUIT
CUIL
PASAPORTE
public normalizar(numero: String): String
public esValido(numeroNormalizado: String): boolean

Tipo y número se validan juntos:
DNI de 7 u 8 dígitos, CUIT/CUIL de 11 dígitos (por su forma: el CSV y el enunciado traen CUIT que no cumplen el dígito verificador), ... El número se guarda sin puntos ni guiones.
Una organización tiene que tener CUIT.
Inválido → DocumentoInvalidoException (400)

estado

EstadosDonante
public ACTIVO: String
public INACTIVO: String
public BLOQUEADO: String
public inicial(): EstadoDonante
public desde(nombre: String): EstadoDonante

BloqueadoState
ActivoState
protected validarTransicion(estadoDestino: String, justificacion: String): void
InactivoState

«abstracta» EstadoDonante
protected validarTransicion(estadoDestino: String, justificacion: String): void

Transiciones válidas (estado inicial: ACTIVO):
ACTIVO → INACTIVO | BLOQUEADO
INACTIVO → ACTIVO
BLOQUEADO → ACTIVO
mismo estado → PersonaConMismoEstadoException
BLOQUEADO sin justificación → FaltaJustificacionException
Una persona BLOQUEADA no registra donaciones (422).

entidad

EntidadBeneficiaria
private razonSocial: String
private direccion: Direccion
private contactos: List<MedioDeContacto>
private representantes: List<Representante>
private campanias: List<Campania>
public cambiarRazonSocial(razonSocial: String): void
public cambiarDireccion(direccion: Direccion): void
public reemplazarContactos(nuevos: List<MedioDeContacto>): void
public reemplazarRepresentantes(nuevos: List<Representante>): void
public agregarCampania(campania: Campania): void
public contactoPreferido(): Optional<MedioDeContacto>
public contarNecesidadesCompatiblesCon(subcategoria: Subcategoria, fecha: LocalDate): int
public recibirDonacion(donacion: Donacion, fecha: LocalDate): int

«embeddable» Direccion
private calle: String
private numero: int
private localidad: Localidad
private codigoPostal: String

«embeddable» Localidad
private nombre: String
private provincia: Provincia

«embeddable» Provincia
private nombre: String

necesidad

NecesidadExtraordinaria

«abstracta» Necesidad
protected id: UUID
protected nombre: String
protected descripcion: String
protected subcategoria: Subcategoria
protected fechaRegistro: LocalDate
protected cantidadObjetivo: int
protected cantidadRecibida: int
public tipo(): TipoNecesidad
public actualizarDatos(nombre, descripcion, cantidadObjetivo, subcategoria): void
public recibir(cantidad: int, fecha: LocalDate): void
public faltanteAl(fecha: LocalDate): int
public cantidadRecibidaAl(fecha: LocalDate): int
public estaSatisfechaAl(fecha: LocalDate): boolean
public esCompatibleCon(buscada: Subcategoria): boolean

«enum» TipoNecesidad
EXTRAORDINARIA
RECURRENTE

Campania
private idCampania: UUID
private idEntidad: UUID {readOnly}
private fechaInicio: LocalDate
private fechaFin: LocalDate
private descripcionGeneral: String
private necesidades: List<Necesidad>
public agregarNecesidad(necesidad: Necesidad): void
public buscarNecesidad(necesidadId: UUID): Optional<Necesidad>
public eliminarNecesidad(necesidadId: UUID): boolean
public estaVigenteAl(fecha: LocalDate): boolean
public necesidadesAbiertasCompatiblesCon(subcategoria: Subcategoria, fecha: LocalDate): List<Necesidad>

periodicidades

NecesidadRecurrente
private periodo: Periodicidad
private fechaInicioPeriodo: LocalDate
public periodoVencido(referencia: LocalDate): boolean
public actualizarPeriodo(hoy: LocalDate): void
public recibir(cantidad: int, fecha: LocalDate): void
public cantidadRecibidaAl(fecha: LocalDate): int

«enum» Periodicidad
SEMANAL(1 semana)
MENSUAL(1 mes)
CUATRIMESTRAL(4 meses)
ANUAL(1 año)
private duracion: Period
public finDelPeriodo(inicio: LocalDate): LocalDate
public inicioDelPeriodoVigente(inicio: LocalDate, fecha: LocalDate): LocalDate

Al entregarse una donación, su cantidad se imputa a las necesidades abiertas compatibles de la entidad, en orden (recibirDonacion).
Una recurrente vuelve a cero al empezar cada período, que avanza de a períodos enteros.
Sin subcategoría, la compatibilidad se busca en el nombre y la descripción.